Quick Sort

AlgoExpert · Hard · Sorting

The problem

- In-place quicksort, recursing into the smaller partition first to bound stack depth.

Intuition

- Partition around a pivot so smaller values go left and larger right, then sort each side.

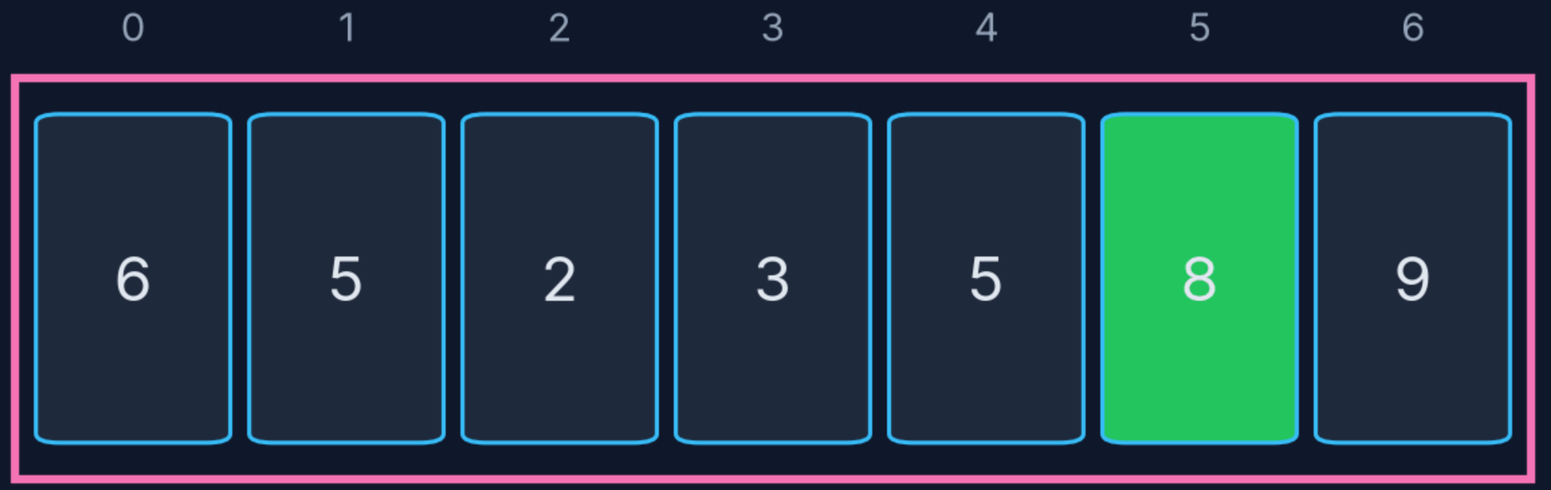
Brute force vs optimal

- Bubble/insertion → $O(n^2)$
- Quick sort → O(n log n) average, O(log n) stack
- Recurse on smaller side first

Approach

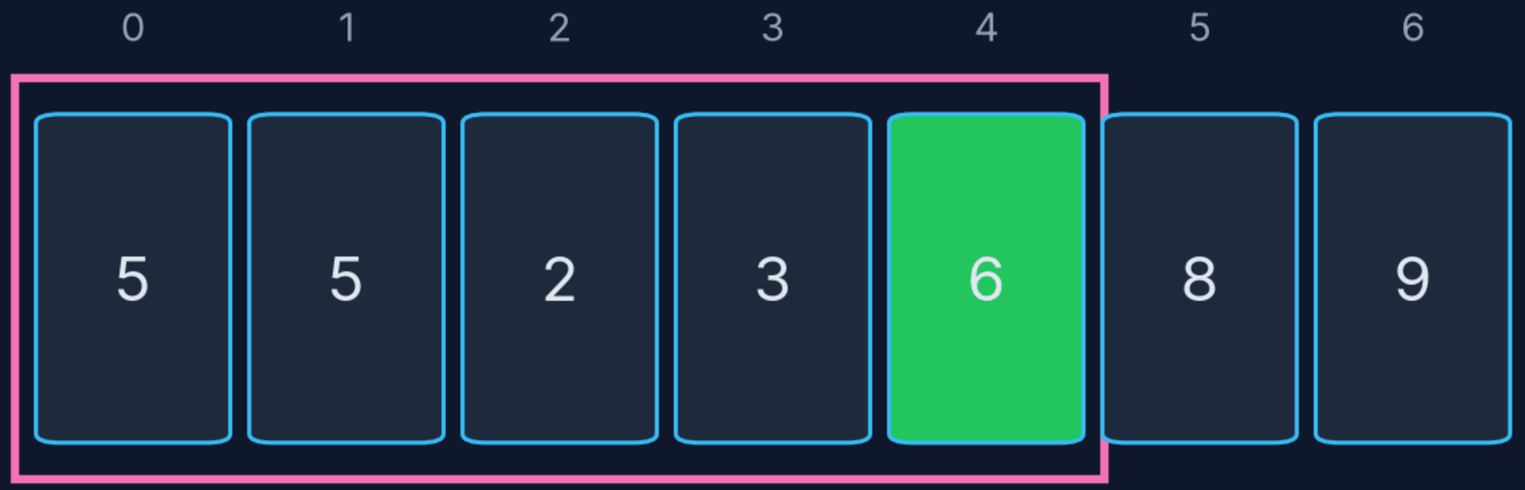
- 1. Pick a pivot
- 2. Partition with two pointers
- 3. Recurse into the smaller side first
- 4. Loop on the larger to bound the stack

Worked example



Pivot 8 placed at 5

Worked example



Pivot 6 placed at 4

Worked example



Pivot 5 placed at 3

Worked example



Pivot 3 placed at 1

Java solution

```java
public class QuickSort {
    public static int[] quickSort(int[] a) { sort(a, 0, a.length - 1); return a; }

    private static void sort(int[] a, int lo, int hi) {
        while (lo < hi) {
            int p = lo, l = lo + 1, r = hi;
            while (l <= r) {
                if (a[l] > a[p] && a[r] < a[p]) swap(a, l, r);
                if (a[l] <= a[p]) l++;
                if (a[r] >= a[p]) r--;
            }
            swap(a, p, r);
            if (r - lo < hi - r) { sort(a, lo, r - 1); lo = r + 1; } else { sort(a, r + 1, hi); hi = r - 1; }
        }
    }

    private static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
}
```

Complexity

- Time: O(n log n) avg, O(n^2) worst
- Space: O(log n)

Edge cases & pitfalls

- Worst case $O(n^2)$ on sorted input → random pivot
- Recurse smaller side to bound stack
- Duplicates

Interview tips & follow-ups

- Mention 3-way partition for duplicates

Recap

- Partition around a pivot so smaller values go left and larger right, then sort each side.
- Complexity: O(n log n) avg, O(n^2) worst